ALF on GPUs: technical report on the port of the auxiliary-field QMC sweep

Branch `gpu-port-unified`, state of 2026-11-13

[author]

2026-11-13

Contents

1 Summary **1**

2 Method in brief **3**

3 Design **4**

4 What is ported **7**

5 Verification **8**

6 Performance **10**

7 Tuning rules **16**

8 Open speedup candidates **18**

9 Limitations and open decisions **19**

A Build lines per platform **21**

B Environment switches that matter in production **22**

1 Summary

This report describes the GPU port of the ALF auxiliary-field quantum Monte Carlo code that lives on the branch `gpu-port-unified`. It is built from the Markdown notes in `docs/gpu/` and from the verification runs of 2026-11-13. Every number comes from one of those notes. The source file is given in a comment next to the number in the LaTeX source. Missing numbers and open items are marked in red as TODO.

What runs on which hardware

One source tree builds for four targets (`README.md`, `BUILD.md`). The default build, `GPU=none`, leaves the CPU code unchanged.

The port was developed for the Hubbard model with a space-time point defect (time-dependent hopping, $N_{\mathrm{FL}} = 2$). It runs the finite-temperature sweep, the local updates (standard, delayed and submatrix), the wraps, the stabilisation and the equal-time and time-displaced measurements on the device. Random numbers, the accept/reject decision, bookkeeping and file output stay on the host. MPI with independent chains and parallel parameter sets (`PARALLEL_PARAMS`) work. Real parallel tempering, the projector code, Langevin, HMC and MALA, other Hamiltonians and several other features are refused at setup, and the run then uses the CPU path (Section 4).

Table 1: Build targets and the hardware on which they were run.

GPU=	Hardware used	Compiler	Libraries
none	any CPU	any ALF compiler	BLAS/LAPACK
nvidia	A30, H100	nvfortran 25.7 / 26.9	cuBLAS, cuSOLVER
intel	Max 1550 (SuperMUC-NG Phase 2)	ifx, oneAPI 2025.x	oneMKL, SYCL shim
amd	MI210	gfortran 14 (amdgcn offload)	hipBLAS, hipSOLVER, HIP kernels

Headline speeds

Table 2 gives seconds per sweep for the Hubbard defect model with $\beta = L$, $\Delta\tau = 0.1$, submatrix update and `Nwrap` 10. Numbers are from different sessions and different timers. Section 6 lists how each was obtained.

Table 2: Seconds per sweep, one chain, and aggregate throughput of the best measured layout at $L = 24$.

Platform and layout	$L = 24$	$L = 32$	Source
A30, one chain	5.25	18.2	`TUNING_DEFECT.md`
A30, 4 ranks with MPS, per chain	10.5	**[TODO: not measured]**	`MPI_GPU.md`, `README.md`
H100, one chain	3.51	9.82	`helma_h100.md`
H100, 8 ranks with MPS, per chain	6.80	34.5	`helma_h100.md`
Max 1550, one chain per tile (V7)	4.95	16.4	`ISSUE_INTEL_VS_A30.md`
Max 1550, 4 chains per tile, per chain	9.3	**[TODO: not measured]**	`ISSUE_INTEL_VS_A30.md`
MI210, one process	4.41	17.5	`amd_mi210/README.md`

The main points:

- At $L = 24$ all four GPUs are within a factor 1.5 of each other for one chain (3.51 to 5.25 s per sweep). The H100 is the fastest at both sizes. At $L = 32$ the H100 is 1.8 times faster than the A30 and the MI210 and Max 1550 are close to the A30 (16.4 to 18.2 s).
- One chain does not fill a GPU. Several chains per GPU raise the throughput by a factor 2 to 4 but slow every chain down by a factor 2 to 3. On the H100 at $L = 24$ eight ranks deliver 1.18 sweeps per second against 0.285 for one rank (Section 6.3).
- Against one CPU thread the A30 is 28 times faster at $L = 24$ and 9.6 times at $L = 16$, and not faster at $L = 12$ (1.76 against 1.68 s). A CPU comparison with a full node for the defect model does not exist **[TODO: CPU node-level number for the defect model]**.
- On Intel, run-time switches of one binary reduced the single-chain time at $L = 24$ from 10.6 to 4.96 s per sweep (Section 6.5). The two last steps, device QR and device `ZUNMQR`, are not yet released for production.
- MPS is required whenever several ranks share an NVIDIA GPU. Without it, two ranks on an H100 are 6.5 times slower per chain at $L = 24$ (Section 6.3).

State of verification

The GPU build is compared with a CPU build of the same compiler, same parameters and same seeds. Agreement is at rounding level, about 10^{-13} with `Mz = .F.` and about 10^{-10} with `Mz = .T.` (Section 5). The consolidation of the six vendor branches into one was checked on 2026-11-13:

- `GPU=none`: four fixed-seed cases are bitwise identical to the pre-merge tip `cc473abc`. All tested upstream features are bitwise identical to upstream ALF master `521ddfca` when `LOBS_ST = 1` is set. Two differences from upstream are intended.
- AMD MI210: bitwise identical to the pre-merge AMD tip `156a0b01` at $L = 6$, 12, 16 and 24; timing unchanged.
- Intel Max 1550 (ifx 2025.3, JIT and AOT builds): against the same-compiler CPU build the deviation is 2.2×10^{-15} and 1.5×10^{-14} ($L = 6$ and 12, `Mz = .F.`) and 5.5×10^{-11} ($L = 6$, `Mz = .T.`). Against the pre-merge Intel build, Green functions, scalars, acceptance and configurations are bitwise identical. With the V7 switches the $L = 6$, `Mz = .T.` value is 1.36×10^{-10}, the same as before the merge, above the tolerance of 10^{-10}.
- NVIDIA A30: the code builds with nvfortran 26.9; the GPU runs are **[TODO: pending]** (card occupied). NVIDIA H100 (Helma): **[TODO: pending]**.

Open points that affect production

The Intel steps V4 and V7 wait for a numerics review of the device QR. The $L = 6$, `Mz = .T.` test fails its pre-registered tolerance with V7 (1.36×10^{-10} against 10^{-10}). The race test of the device QR cannot detect a race on the `tau` slot. The staging-buffer setting, worth 33 % at $L = 24$ on Intel, is not in the production job script of the running set. Section 9 lists these and the decisions that belong to the owner.

2 Method in brief

2.1 The sweep

ALF samples a Hubbard–Stratonovich auxiliary field on a space-time lattice with N sites and $L_{\mathrm{trot}} = \beta/\Delta\tau$ time slices. For a fixed field the fermions are integrated out. The weight and all equal-time Green functions follow from products of $N \times N$ matrices (`PROFILE.md`, `HARD.md`). One *sweep* goes up and then down through all slices. In the GPU documents it includes the measurements (`TUNING_DEFECT.md`).

On each slice the code does the following.

1. **Local updates.** For every site it proposes a field change, computes the ratio of weights from the Green function G, accepts or rejects, and on acceptance updates G by a rank-one matrix. The rank-one update is memory-bound BLAS-2 work.
2. **Wrap.** It moves G from slice τ to $\tau + 1$ by multiplying with the hopping matrix and the interaction factor.
3. **Stabilisation.** Every `Nwrap` slices the product of the B matrices is re-factorised with a QR decomposition, and G is rebuilt (`CGR`). Between these points the products are numerically unstable.
4. **Measurements.** Equal-time observables are accumulated from G on every slice. Time-displaced observables (`Ltau = 1`) need an extra pass over τ with a $2N \times 2N$ factorisation (`CGR2_2`).

2.2 Where the time goes on a CPU

Table 3 is the profile of the upstream CPU code (stack sampling, one rank, one thread; `PROFILE.md`). It is for the plain Hubbard model, $U = 4$, $\beta = 4$, $\Delta\tau = 0.1$, not for the defect model.

Table 3: Share of wall time per group of routines on one CPU core, in percent (`PROFILE.md`). Hubbard square lattice, $U = 4$, $\beta = 4$, `Ltau = 1`. cb: checkerboard hopping, dense: dense hopping matrix.

	cb			dense		
Group	12^2	20^2	32^2	12^2	20^2	32^2
Stabilisation (CGR, CGR2_2, decompose)	47.0	55.4	42.3	32.0	32.0	26.4
Local updates (Upgrade2, Op_Wrap)	22.5	25.3	34.9	12.8	16.2	21.0
B products (wraps, propagators, hopping)	12.1	5.6	13.3	41.6	42.4	42.0
Measurements	16.9	11.5	8.1	11.7	6.3	5.1
Rank-one update (`ZGERU`) alone	17.2	23.5	33.3	9.7	15.2	20.1
Wall time per sweep, 1 rank (s)	0.60	8.0	193	0.80	13.7	294

Three points follow for the port (`PROFILE.md`, `BENCHMARKS.md`):

- Stabilisation is the largest group. Its QR with column pivoting has no good batched GPU equivalent.
- The share of the rank-one update grows with N (17 to 33 % for checkerboard hopping). It is memory-bound, and one GPU launch per accepted flip is slow. This is the case for delayed and submatrix updates.
- Measurements are not negligible (17 % at $N = 144$) and are loops with scatter-adds. They have to run on the device, otherwise G must be copied back three times per slice.

The measured cost of one sweep on one core of the `nividia` host (Zen 3, 16-core EPYC 7763 VM) for the same model is 0.60 s at $N = 144$ and 193 s at $N = 1024$ with checkerboard hopping (`BENCHMARKS.md`). The values at $N = 1024$ come from profiling runs with other jobs on the node and are indicative only.

2.3 Delayed and submatrix updates

A GPU cannot do rank-one updates fast. Kernel benchmarks on the A30 give 12 to 15 μs for one OpenMP target region and 28 to 34 μs for a rank-one update with a host decision at $N \leq 400$ (`BENCHMARKS.md`,

ADR-001-programming-model.md). A rank-k update with $k = 128$ costs 0.21 to 1.25 μs per update on the device, 25 to 40 times less than rank one. This motivates two schemes of ALF (DELAYED_SUBMATRIX.md):

Delayed update. $G = G_0 - XY^T$ is kept in low-rank form for up to N_delay accepted flips and flushed with one ZGEMM.

Submatrix update. Between flushes only the diagonal wrap scalings change G_0. The host copies the principal block $G_0(B, B)$ of one block of N_delay consecutive vertices once and decides all proposals of the block from it. At the block end the device applies the scalings and flushes with two ZGEMM calls.

On the A30 the block mode brought the submatrix scheme on a 20×20 lattice (six sweeps) from 49.7 s to 8.7 s. That is 4.9 times faster than the earlier standard run (42.2 s). Acceptance is identical (DELAYED_SUBMATRIX.md). At $L = 16$ to 32 it is 2.4 to 5 times faster than the standard update (TUNING_DEFECT.md). The block mode replaced about 8 synchronous device operations per site by about 12 per block.

2.4 Point defect and time-dependent hopping

The production model is the Hubbard model with a space-time point defect (TIMEDEP_HOPPING.md). A hopping element depends on the time slice through a function g_t. The element is applied as $A(g) = U \exp(gE)U^\dagger$ and is skipped when $|g|^2$ is below a threshold. For the point defect the pin is at one slice, $g_t(1) = -1$ ($\tau = 0$), after the merge of 2026-11-09. The static hopping is a dense $N \times N$ matrix. The Checkerboard flag is ignored when ham_h0 $\neq 0$.

Two consequences matter for the GPU. First, a time-dependent element has one nonzero eigenvalue in the defect case, so it is stored on its support (one site) instead of as a dense matrix. Without this, a set of matrices at $L = 32$ needs more than 2 GB. Second, the down sweep measures all slices, including $\tau = 0$, and SpinZ_localtau has $L_{\mathrm{trot}} + 1 - 2T_{\mathrm{htrot}}$ columns ($\tau = 0$ to β). The default LOBS_ST is 0; upstream ALF measures $2L_{\mathrm{trot}} - 1$ times per sweep and the unified branch measures $2L_{\mathrm{trot}}$ times (Section 5).

3 Design

The design decisions are in two architecture decision records, accepted on 2026-10-27 after the kernel benchmarks of Phase 1 (ADR-001-programming-model.md, ADR-002-alf-la-and-data-model.md).

3.1 Programming model

ALF is Fortran with class-based Hamiltonians. One source tree has to run with nvfortran, ifx and gfortran, and GPU=none has to stay upstream ALF. The decision (ADR-001) is:

- **OpenMP 5.x target offload** with one conservative subset that both nvfortran and ifx accept: target enter/exit data, target update, target teams distribute parallel do on explicit-shape contiguous arrays, and use_device_addr. No declare mapper, no unified memory, no polymorphism or allocation in device code.
- **Vendor libraries behind one layer, alf_la.** Large matrix work (GEMM, QR, LU, TRSM) goes to cuBLAS/cuSOLVER, oneMKL or hipBLAS/hipSOLVER. Nothing outside alf_la knows the backend.
- **Selection at configure time**: GPU=none|nvidia|intel|amd.
- **Same trajectory as the CPU.** The update is organised as propose, ratio, decide, apply or defer, flush. The decision is made on the host with the same random-number order, so that a GPU run follows the CPU run flip by flip for the same compiler and seeds.

OpenACC, CUDA Fortran, do concurrent offload and a SYCL or Kokkos rewrite were considered and not adopted (ADR-001). The only measured difference between OpenMP and OpenACC on the A30 is the asynchronous launch: about 10 μs for OpenMP nowait and 2.6 μs for OpenACC async (BENCHMARKS.md). With a device-side decision per site OpenACC gives 6.5 to 8.8 μs per site against 15 to 18 μs for the host decision at $N \leq 400$. The lever turned out to be algorithmic (delayed and submatrix updates), which removes launches in either model.

The kernel benchmarks (BENCHMARKS.md) divide the work in two parts. Bulk work, such as stabilisation, dense products and fused element-wise wraps, is 5 to 40 times faster than one core at $N \geq 400$. Per-site work is launch-bound: an OpenMP target region costs 12 to 15 μs, a cuBLAS call 5 μs and a scalar device-host-device round trip 16 to 23 μs.

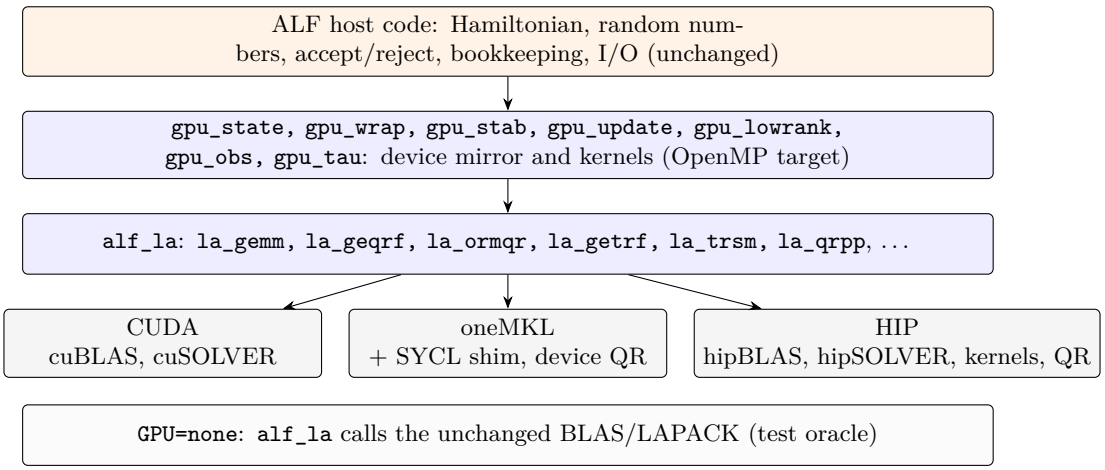


Figure 1: Layers of the port. Upstream routines get at most a guarded early branch into the GPU modules (ADR-002).

3.2 Layers

3.3 alf_la

`alf_la_mod.F90` is one module with plain-array routines in LAPACK argument order: `la_gemm`, the real-times-complex variants for dense hopping, strided batched `GEMM` over flavours, `la_geru`, `la_geqrf`, `la_ungqr`, `la_ormqr`, `la_getrf`, `la_getrs`, `la_trsm`, and `la_qrpp`, which is a column-norm pre-pivot followed by an unpivoted QR and returns a pivot vector compatible with `ZGEQP3` (ADR-002). No array descriptors or derived types cross the interface. The arrays are device-resident and `alf_la` gets their device addresses through `use_device_addr`.

Library calls must run on the queue that the OpenMP runtime uses for the kernels, otherwise kernels and library calls are not ordered. This was tested and amended in ADR-002:

- **CUDA.** A stream created by `la_init` is not ordered against nvfortran's target regions. Synchronous regions use the per-thread default stream, so the cuBLAS and cuSOLVER handles are set to the stream returned by `acc_get_cuda_stream(acc_async_sync)`. As a consequence `nowait` and `depend` regions are not ordered against library calls. The test `t_order` checks this. The build flag `-DALF_LA_SYNC_ALWAYS` brackets every call with a sync.
- **oneMKL.** BLAS goes through `!$omp dispatch`. The LAPACK factorisations go through a small SYCL shim on the interop queue of the OpenMP runtime.
- **CUDA reproducibility.** cuSOLVER runs in deterministic mode and cuBLAS atomics are off (`RUNTIME_SWITCHES.md`).

3.4 Device-resident data

`gpu_state_mod.F90` allocates and maps one flat mirror at setup (`target enter data`) and frees it at the end. Nothing is allocated in loops. Table 4 lists the main arrays (ADR-002).

Table 4: Main arrays of the device mirror (ADR-002).

Array	Shape	Content
`G`	(N, N, N_{FL})	Green function, on the device for the whole run
`cb_idx, cb_exp`	per family	checkerboard bond pairs and 2×2 exponentials
`T_exp`	$(N, N, N_{\mathrm{FL}}, 4)$	$e^{\pm T}$, $e^{\pm T/2}$ for dense hopping
`V_site, V_exp`	per `Op_V`	sites and exponential table of the diagonal vertices
`nsigma_d`	$(N_{\mathrm{op}}, L_{\mathrm{trot}})$	auxiliary field
`U, D, V` work, `udvst_d`	per stabilisation point	stabilisation stack
`GT0, G0T, GTT`	(N, N, N_{FL})	time-displaced pass, only for `Ltau = 1`

The stack dominates memory: peak $\approx (4N_{\mathrm{STM}} + 40) \cdot 16\,N^2$ bytes plus 0.3 GB, with $N_{\mathrm{STM}} = L_{\mathrm{trot}}/\mathtt{Nwrap}$ (`TUNING_DEFECT.md`). The formula gives 19.7 GB at $L = 48$, $\beta = 48$ and the measurement is 19.6 GB. A card with 24 GB holds $L = 48$; for $L = 64$ the formula gives 45 GB at $\beta = 32$, but $L = 64$ cannot be started at present (Section 9).

3.5 Backends

CUDA (NVIDIA). cuBLAS and cuSOLVER through `bind(C)`. The device of rank r is $r$mod the number of devices. Several ranks per GPU need an MPS daemon (Section 6.3).

oneMKL and SYCL (Intel). Beyond the shim, three items were added during the SuperMUC-NG work (`ISSUE_INTEL_VS_A30.md`):

- an *async block path* (`libalfsb.so`): the block kernels of the submatrix update are submitted on the OpenMP interop in-order queue without a wait (4.3 μs per operation against 12.3 μs synchronous);
- an own *device Householder QR* (`libalfdevqr.so`, loaded with `dlopen`) that replaces the hybrid host-device `geqrf` of oneMKL, in two versions, plus a device `ZUNMQR`;
- explicit-shape dummy arguments and a fused kernel in the submatrix update, which cut first-private arrays from 168 800 to 28 544 per sweep.

HIP (AMD). The AMD build uses gfortran 14 with amdgcn offload. Its libgomp runtime has a high launch cost (`amd_mi210/README.md`). The AMD backend therefore replaces the small OpenMP kernels by HIP kernels (copies, block gather and scale, flush gather, scatter, correlations, scalar and local observables, stabilisation helpers) and adds its own blocked Householder QR and Q-apply. A patched libgomp GCN plugin pools kernel arguments and signals, and calls a presync hook so that HIP calls need no `hipDeviceSynchronize` after each launch.

3.6 What stays on the host

Random numbers, the accept/reject decision, field bookkeeping, bin I/O, the Hamiltonian setup and all of ALF outside the GPU path stay on the host. With the submatrix scheme the host decides all proposals of a block from a copy of the principal block of G_0 (`DELAYED_SUBMATRIX.md`). Under `STABLOG` the CGR factorisations run as host `ZGEQP3` with copies of the matrices, because no GPU-friendly pivoting met the precision criteria (ADR-001). The time-displaced observables with the defect still use the host `ObserT` and copy four matrices per τ slice (`TIMEDEP_HOPPING.md`).

3.7 Activation, fallback and guard

At setup `gpu_supported` checks the Hamiltonian, the vertices and the options. If a check fails, the whole run uses the CPU path and `info` gives the reason. It never gives a silent wrong answer. A GPU run should show `GPU path (gpu_state): ACTIVE` and `GPU observables: device equal-time measurement ACTIVE` in `info`. All runs set `OMP_TARGET_OFFLOAD=MANDATORY`, otherwise a failed device initialisation silently runs on the CPU.

A production guard (`RUNTIME_SWITCHES.md`) reads the environment: any `ALF_GPU_*` or `ALF_LA_TEST_*` variable except `ALF_GPU_DISABLE` and `ALF_GPU_DEBUG` stops a run unless it is declared a debug run. The variables `ALFSH_*`, `ALF_HIP_*` and `ALFDEVQR_*` are outside the guard and have to be checked by hand.

3.8 Compiler problems met

Table 5: Compiler and runtime problems that shaped the code or the build.

Toolchain	Problem and handling
nvfortran 26.9	`blk_end` is mis-compiled at `-O2` and above (wrong G after the first stabilisation). It is pinned to `-O1` with `!pgi$r opt=1`. The cause is not understood (`DELAYED_SUBMATRIX.md`).
nvhpc 24.11	The GPU build compiles but the $L = 8$ run aborts with `la_qrpp: NaN or Inf`. The CPU run of the same compiler is fine. Not used (`helma_h100.md`).
nvhpc 25.7	Needs `-gpu=cc90,cuda12.9`; without it the default CUDA 13 device compile fails (`helma_h100.md`).
ifx $\leq$ 2025.3	`omp_get_interop_int/ptr` are missing in `omp_lib`, so `alf_la_mod.F90` declares them with `bind(C)` (`intel_pvc_supermuc_ng2.md`).
oneAPI 2025.3	Compiles, but the OpenMP-to-SYCL interop crashes at run time with the Level Zero V2 adapter. 2025.1 works and is used (`intel_pvc_supermuc_ng2.md`).
ifx (OpenMP tasks)	`nowait`/`depend` target tasks run undeferred; no gain (gate A: 97 to 103 % of the baseline time) (`ISSUE_INTEL_VS_A30.md`).
gfortran 14 (amdgcn)	Launch cost of the libgomp GCN plugin is high: 253 μs for an empty target region, 37 μs with the patched plugin (`amd_mi210/README.md`).

3.9 Source layout

The device code is in `Prog/gpu_*_mod.F90` (state, wrap, stabilisation, update, low-rank blocks, equal-time and time-displaced observables). The backends are in `Libraries/Modules/`: `alf_la_mod.F90` (interface and CUDA/oneMKL/host code), `alf_la_sycl_*` (Intel shim, async block path, device QR) and `alf_la_hip_mod.F90`, `alf_hipk_mod.F90`, `alf_hip_*.cpp` (AMD). Unit tests of `alf_la` and the acceptance harness are in `testsuite/alf_la.tests/` and `testsuite/gpu_reference/` (`README.md`).

4 What is ported

Table 6 lists the features per backend. The base is `FEATURES.md` (state 2026-11-06). Rows on MPI and parallel parameters are updated from `MPI_GPU.md`, `PP_GPU.md`, `helma_h100.md`, `intel_pvc_supermuc_ng2.md` and the SuperMUC session notes, which are newer. **[TODO: `FEATURES.md` is stale: it has the Arc B580 as the Intel GPU, lists MPI and tempering as refused on Intel and has no AMD column. It should be updated; the table here follows the newer notes]**

Entries: S = supported and validated, R = refused at setup (the run uses the CPU path), N = not implemented (no GPU code and no check, a CPU-only feature that the model does not use), ? = not documented. The AMD notes do not list feature coverage; the AMD column shows only what the notes state. **[TODO: AMD feature coverage is not documented; fill the ? entries after a check of `gpu_supported` on the HIP build]**

Table 6: Feature coverage of the GPU path. NVIDIA: A30 and H100. Intel: Max 1550 (the first notes were for an Arc B580).

Feature	CPU	NVIDIA	Intel	AMD
Finite-T DQMC, Hubbard (predefined)	S	S	S	S
Projector ($T = 0$)	S	R	R	?
Other Hamiltonians (Plain_Vanilla, Kondo, tV, Spin_Peierls, Z_2, LRC)	S	R	R	?
HS fields type 1, 2 (diagonal, rank-1)	S	S	S	?
HS fields type 3, 4; non-diagonal or rank > 1 vertices	S	R	R	?
Local updates, standard	S	S	S	?
Delayed and submatrix updates	S	S	S	S
Cross-slice delay (`Lowrank_cross_slice`)	S	R	R	?
Global-in-τ and space-time moves	S	R	R	?
Langevin, HMC, MALA	S	R	R	?
Parallel tempering (exchange)	S	R	R	?
Parallel parameter sets (`PARALLEL_PARAMS`), no exchange	S	S	S	?
MPI, independent chains	S	S	S	?
Checkerboard, dense hopping, `Symm`	S	S	S	?
Time-dependent hopping (point defect)	S	S	S	S
Time-dependent couplings of `Op_V`	S	R	R	?
Flavour symmetry reconstruction	S	R	R	?
Stabilisation default (pre-pivoted QR)	S	S	S	S
Stabilisation `LOG` (host QR with pivoting)	S	S	S	?
`STAB1/2/3`, `QRREF`, `Propose_S0`	S	R	R	?
Equal-time observables (Hubbard)	S	S	S	S
Time-displaced observables (`Ltau = 1`)	S	S	S	?
Custom observables, entanglement	S	N	N	N
HDF5 output (host side)	S	S	S	?

Remarks on the table

- The GPU path is taken only if every check passes. Otherwise the whole run uses the CPU path, and `info` states the reason (`FEATURES.md`).
- The time-displaced pass for the defect runs its propagation on the device, but the τ observables with the defect still use the host `ObserT` (`TIMEDEP_HOPPING.md`). `Ltau = 0` is recommended; `Ltau = 1` works but is slower (`TUNING_DEFECT.md`).
- Time-dependent hopping elements are accepted only if they follow all static elements in `Op_T(:,nf)`. Oth-

erwise the run is refused with a reason (`TIMEDEP_HOPPING.md`).
- Tempering is refused only as real tempering. Since the change of `gpu_state_mod.F90` (`PP_GPU.md`), builds with `PARALLEL_PARAMS` are accepted and `Exchange_Step` is skipped in `main.F90`.
- On one Max 1550 tile the standard and delayed schemes were also timed at $L = 24$: 18.32 and 19.66 s per sweep against 5.01 s for the submatrix scheme (`ISSUE_INTEL_VS_A30.md`, `p7/NOTES.md`).

Not implemented, by group (`NOT_IMPLEMENTED.md`)

Model and feature coverage. Rank > 1 and non-diagonal vertices (would open tV, Kondo, Spin_Peierls); flavour-symmetry reconstruction (Plain_Vanilla, Spin_Peierls); device observables for other models, entanglement and custom observables; HS fields of type 3 and 4; global moves; `Propose_S0`; time-dependent couplings of `Op_V`; `STAB1/2/3` and `QRREF`; cross-slice delay; parallel tempering; projector; Langevin and HMC.

Performance at small and medium N. Device-side accept decision, start-up cost, per-bin output, remaining launch overhead.

Performance at large N. Sparse `Symm`, a memory footprint of about $1650\,N^2$ bytes, block structure of the τ pass, Cholesky-QR variants.

Size limits

At $\beta = 5$ a 62×62 lattice fits an A30 (23 967 MiB) and 63×63 does not (`DELAYED_SUBMATRIX.md`). In the defect setting with $\beta = L$ the peak memory is 3077 MiB at $L = 32$ and 19 647 MiB at $L = 48$ (`TUNING_DEFECT.md`). The H100 uses 3584 MiB per rank at $L = 32$ (`helma_h100.md`).

5 Verification

5.1 Method

Same-compiler CPU reference. A GPU build is compared with a CPU build (`GPU=none`) made by the same compiler, with the same parameters and the same seeds. The CPU reference must come from the same compiler because the random-number stream of the GNU build differs from that of nvfortran, so trajectories from different compilers are not comparable (`TIMEDEP_HOPPING.md`). With the same seeds and the host decision of ADR-001 the GPU trajectory follows the CPU trajectory flip by flip until rounding differences accumulate. The driver is `scripts/defect/run_cmp.py` (`VERIFICATION.md`, `README.md`).

Deviation measure. For each output file the deviation is $\max|A - B| / \max|A|$ over the file. Acceptance rates and final configurations have to be identical.

Tolerances. About 10^{-13} for `Mz = .F.` and about 10^{-10} for `Mz = .T.` (`README.md`). The Intel session used the same two values; the SuperMUC first session used 10^{-8} as the criterion (`intel_pvc_supermuc_ng2.md`). The `Mz = .T.` deviations are the intrinsic accuracy of G and not a code difference: the same code gives 10^{-14} with `Mz = .F.` (`TIMEDEP_HOPPING.md`). Production runs additionally monitor `Precision Green`; the requirement in the Helma benchmarks is below 10^{-8} (`helma_h100.md`).

Test standard (`VERIFICATION.md`, from 2026-10-29). The main rules are:

- Acceptance runs use binaries built from `git archive` of a frozen candidate. The committed tree equals the accepted tree except for `docs/**`.
- Every run writes a provenance record (candidate hash, binary hash, compiler, environment, parameter and seed hashes, output hashes). Debug and fault variables are refused unless the acceptance file declares a test run. GPU runs have to show the `ACTIVE` lines.
- Each ticket has a pre-registered acceptance file with gates, thresholds, seeds and positive controls. Changing it needs the owner's dated approval and fresh seeds.
- Verdicts are computed by a script. NaN, missing files and inactive GPU paths give `INPUT_ERROR`, never a pass.
- Every gate has a positive control that must fail, for example a scaled tau dataset, `ALF_GPU_NS=0` or a planted 10^{-12} perturbation. Statistical gates carry a null control (CPU against CPU).
- Tiers: T0 (every iteration), T1 (implementer, frozen candidate, about 90 min GPU time) and T2 (manager, fresh seeds, about 120 min, plus an independent verifier).

Determinism. Where bit identity holds (same binary run twice, same commit rebuilt with the same toolchain), it is demanded. On 2026-10-29 the `LOG` path on the A30 was bitwise reproducible, the default path was not (20 of 38 datasets differed, first in `Den_eq/back` at about 10^{-12} relative). MPS and CPU jobs

alongside did not change results (`VERIFICATION.md`). **[TODO: state of ticket 212 (reproducibility of the default path); the notes do not say whether it is closed]**

5.2 Earlier results against the same-compiler CPU reference

Table 7 collects the same-seed results from the platform sessions before the merge.

Table 7: Worst relative deviation, GPU against the same-compiler CPU build, same seeds. Acceptance identical in all rows.

Platform	Case	Deviation	Source
A30, nvfortran 26.9	$U = 0$, $L = 4$ and $L = 8$, all files	3.0×10^{-14}, 8.5×10^{-14}	`TIMEDEP_HOPPING.md`
A30	$L = 4$, `Mz = .F.`, standard, delayed, submatrix	1.7, 1.5, 2.2×10^{-15}	`TIMEDEP_HOPPING.md`
A30	$L = 8$, `Mz = .F.`, three schemes	8.4×10^{-15}	`TIMEDEP_HOPPING.md`
A30	$L = 8$, `Mz = .T.`, three schemes	5.1×10^{-10} (largest in `SpinZ_localtau`)	`TIMEDEP_HOPPING.md`
A30	$L = 32$, $\beta = 4$, 1 sweep, `Mz = .F.` standard, delayed	6.0×10^{-15}, 1.4×10^{-14}	`TIMEDEP_HOPPING.md`
A30	one MPI rank against non-MPI, $L = 16$	10^{-12} level (`Ener_scal` 7×10^{-12}, `SpinZ_localtau` 1.4×10^{-12})	`MPI_GPU.md`
H100, nvhpc 25.7	$L = 8$, `Mz = .F.`, N_{delay} 64	5.2×10^{-13}	`helma_h100.md`
Max 1550, ifx	`Mz = .F.`, $L = 6$ to 12, N_{delay} 8 to 128	3.0×10^{-15} to 2.7×10^{-14}	`intel_pvc_supermuc_ng2.md`
Max 1550, ifx	`Mz = .T.`, $L = 6$ ($L = 12$, N_{delay} 128)	2.5×10^{-11} (3.4×10^{-9})	`intel_pvc_supermuc_ng2.md`
Max 1550, V4	`Mz = .F.` $L = 6$, $L = 12$; `Mz = .T.`	2.2×10^{-15}, 1.6×10^{-14}; 6.0×10^{-11}	`ISSUE_INTEL_VS_A30.md`
Max 1550, V7	`Mz = .F.` $L = 6$, $L = 12$; `Mz = .T.` $L = 6$	2.3×10^{-15}, 1.6×10^{-14}; 1.36×10^{-10} (fails 10^{-10})	`ISSUE_INTEL_VS_A30.md`

For V7 the criterion is noise-limited. oneMKL itself fails the $L = 6$, `Mz = .T.` tolerance for 2 of 5 seed sets (1.13×10^{-10} and 1.84×10^{-10}). A new pre-registered rule needs the owner's approval (`ISSUE_INTEL_VS_A30.md`).

Physics and statistics. Delayed and submatrix reproduce the standard update bin by bin on the A30 and on the B580 and agree with exact diagonalisation of a four-site chain within one standard deviation (`DELAYED_SUBMATRIX.md`). The upstream `test_vs_ed` Hubbard chain passes with GNU, ifx and nvfortran (deviations of 0.64, 1.48 and 1.52 standard deviations, `BASELINE.md`).

Precision of G at production sizes. `Precision Green` (mean / max) at $L = 24$, 28, 32 on the Max 1550 is $1.7 \times 10^{-14}/1.2 \times 10^{-11}$, $4.6 \times 10^{-14}/1.1 \times 10^{-10}$ and $1.4 \times 10^{-13}/5.3 \times 10^{-10}$ (`intel_pvc_supermuc_ng2.md`). On the H100 at $L = 32$ the maximum is 1.4×10^{-10} to 7.2×10^{-10} in all configurations (`helma_h100.md`).

5.3 Verification of the consolidated branch, 2026-11-13

`gpu-port-unified` merges six vendor lines (`CONSOLIDATION.md`). The checks below test that the merge changed nothing it should not have changed. They were run on 2026-11-13. Entries that were not finished when this report was written are marked **[TODO: pending]**. Source of the Intel and AMD rows: the verification table of `CONSOLIDATION.md`.

Upstream features tested for bitwise identity against ALF master `521ddfca`, with `GPU=none`, gfortran, $L = 4$ and `LOBS_ST = 1`: Hubbard with `Mz` true and false, `Symm`, checkerboard, `Ltau` 0 and 1, projector, `N_SUN`, continuous fields, Langevin, HMC, MALA, Kondo, tV and Plain Vanilla.

Two intended differences from upstream.

1. The default `LOBS_ST` is 0. The down sweep then also measures at $\tau = 0$, which gives $2L_{\text{trot}}$ instead of $2L_{\text{trot}} - 1$ measurements per sweep. With `LOBS_ST = 1` the upstream behaviour is restored.
2. `ham_h0` now means the space-time point defect and not upstream's static pinning field.

Limits of these checks. The `GPU=none` checks show that the merge did not change the CPU path. They say nothing about the device code, which was checked on the MI210 and on the Max 1550. The two pending NVIDIA rows (A30, H100) have to be run before the consolidated branch can be called verified on NVIDIA.

Table 8: Checks of the consolidated branch. Source of all entries: verification runs of 2026-11-13.

Platform, build	Comparison	Result	Status
macOS, gfortran 16, `GPU=none`	4 fixed-seed cases ($L = 4/6$, `Mz` T/F, delayed/submatrix) against the pre-merge tip `cc473abc`	bitwise identical	done
macOS, gfortran, `GPU=none`, $L = 4$	upstream ALF master `521ddfca`, `LOBS_ST = 1` set explicitly	bitwise identical for all tested upstream features (list below)	done
`GPU=none`, gfortran	delayed and submatrix against the standard update	agree to 10^{-12} with identical trajectories	done
`GPU=none`, gfortran	`SpinZ_localtau` against an exact $U = 0$ calculation	agrees to 4.7×10^{-14}	done
AMD MI210, `GPU=amd`, gfortran 14 GCN + HIP	unified against pre-merge tip `156a0b01`, $L = 6$, 12, 16, 24; default, `ALF_HIP_ASYNC=0`, `ALF_HIP_QR=0`	bitwise identical in all cases	done
AMD MI210	GPU against same-compiler CPU	1.15×10^{-14} ($L = 6$), 5.0×10^{-14} ($L = 12$)	done
AMD MI210	timing	unchanged, about 4.5 s per sweep at $L = 24$	done
NVIDIA A30, nvfortran 26.9	build of GPU and CPU reference	builds	done
NVIDIA A30	GPU runs	**[TODO: pending]** (card occupied by a production run)	pending
NVIDIA H100 (Helma)	GPU runs	**[TODO: pending]**	pending
Intel Max 1550 (SuperMUC-NG Phase 2), ifx 2025.3, JIT and AOT	GPU against same-compiler CPU, default switches	2.2×10^{-15} ($L = 6$, `Mz = .F.`), 1.5×10^{-14} ($L = 12$, `Mz = .F.`), 5.5×10^{-11} ($L = 6$, `Mz = .T.`)	done
Intel Max 1550, V7 switches	GPU against same-compiler CPU	2.3×10^{-15}, 1.6×10^{-14}, 1.36×10^{-10} ($L = 6$, `Mz = .T.`; same value as before the merge, tolerance 10^{-10}, fails)	done, one failure
Intel Max 1550	unified against pre-merge Intel build (p5)	Green functions, scalars, acceptance and configurations bitwise identical; `Den_eq`, `SpinT_eq`, `SpinZ_eq` differ by at most 3.3×10^{-16} (expected, `acc0` parallel reduction from pp-opt)	done
Intel Max 1550, $L = 24$, one chain, V7	timing, before and after the merge	4.92 to 4.98 s per sweep before, 4.42 to 4.48 s after; observables 1.18 to 0.74 s per sweep; updates 1.43, stabilisation 0.81, CGR 1.23 s unchanged	done

The Intel jobs 560852 to 560860 used about 0.85 node-hours. The CI (`.github/workflows/tests.yml`) builds and tests the CPU path only (`README.md`). The `acc0` parallel reduction that arrived from the pp-opt line changes the summation order of the `acc0` terms. On the Max 1550 the effect is at most 3.3×10^{-16} in `Den_eq`, `SpinT_eq` and `SpinZ_eq`, and it cuts the observables phase from 1.18 to 0.74 s per sweep, about 10 % of the sweep (`CONSOLIDATION.md`).

6 Performance

6.1 How the numbers were obtained

The numbers come from different sessions on different machines. Table 9 states how each was measured. They are comparable to within the differences stated there, not better.

6.2 Time per sweep against L

Figure 2 shows the seconds per sweep for one chain against L with $\beta = L$. For the A30 the full scan of `TUNING_DEFECT.md` is shown. The filled squares use the best `N_delay` of the scan and enter the fit. The open squares use `N_delay` 4 or 8 and are therefore too slow. The dashed lines are least-squares fits of $t = aL^b$ in log-log space to

Table 9: Measurement method per platform. All runs: Hubbard defect model, $N_{\mathrm{FL}} = 2$, $h_0 = 1$, `Mz = .F.`, $\beta = L$, $\Delta\tau = 0.1$, `Nwrap` 10, submatrix, unless stated.

Platform	Method
A30 (`nividia`)	nvfortran 26.9, one process, 2 sweeps, timer in the run, excludes the 1 to 2 s mirror set-up (`TUNING_DEFECT.md`). The older multi-rank numbers are CPU time of rank 0 divided by the number of sweeps and include start-up. Whether MPS was active in those runs is not recorded (`helma_h100.md`).
H100 (Helma)	nvhpc 25.7, 6 sweeps per test (`NSweep` 2, `NBin` 3), (finish time − start time)/6 of rank 0. Excludes the 0.3 to 0.6 s mirror set-up, includes the first-sweep warm-up, so it is slightly pessimistic. One MPS daemon, thread share 25 % (`helma_h100.md`).
Max 1550 (SuperMUC-NG Phase 2)	ifx and oneMKL 2025.x, in-loop timers, variants alternated in the same job. V4 and V7 were also measured in different jobs (`ISSUE_INTEL_VS_A30.md`). One chain on one tile; the card layout uses 32 ranks per node, 4 per tile, with CCS 4.
MI210	gfortran 14 and HIP, steady state from the difference of 2- and 4-sweep runs (`amd_mi210/README.md`).

the filled points of each platform (data in `data/sweep_*.dat`).

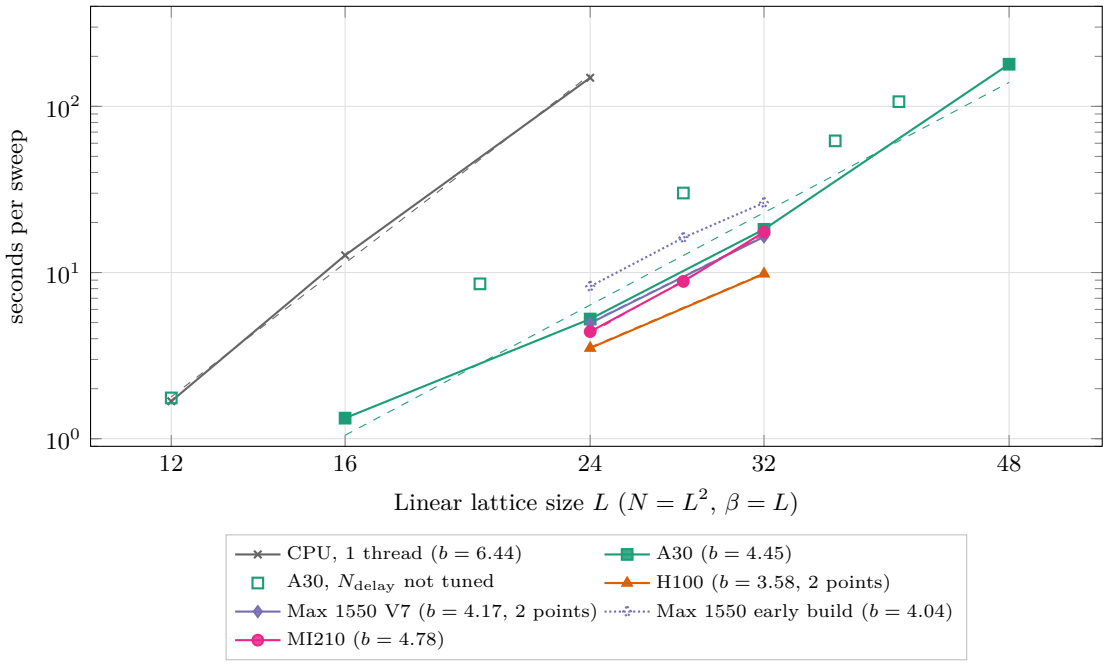


Figure 2: Seconds per sweep for one chain against L. Sources: `TUNING_DEFECT.md` (A30, CPU), `helma_h100.md`, `ISSUE_INTEL_VS_A30.md` (V7), `intel_pvc_supermuc_ng2.md` (early build), `amd_mi210/README.md`. The exponent b in the legend is from the fit of the dashed line. The CPU is one thread of the `nividia` host with the same source (`GPU=none`).

Table 10 gives the numbers behind the figure and the local exponent between neighbouring points.

Three remarks:

- The exponents are well above the 3 of a single dense matrix product at fixed β. The work per sweep grows with $L_{\mathrm{trot}} \propto L$ slices and with the $N \propto L^2$ cost per slice of both the update blocks and the N^3 stabilisation. A fit with two to four points does not separate these contributions. The H100 and Max 1550 fits rest on two points and are indicative only.
- The local exponents rise with L on the A30 and the MI210, but not on the early Max 1550 build. For the A30 the local exponent grows from 3.4 to 5.6. The MI210 value between $L = 28$ and 32 (8.84 to 17.5 s, a factor 2.0 for a size ratio of 1.14) is unusually steep. **[TODO: check the MI210 value at $L = 28$: an earlier note in the same file gives 16.6 s per sweep at $L = 28$ for the version before the QR work]**
- Below $L = 16$ the GPU does not win: at $L = 12$ the A30 needs 1.76 s with `N_delay` 4 and the one-thread CPU 1.68 s. At $L = 16$ the speed-up is 9.6, at $L = 24$ it is 28 (`TUNING_DEFECT.md`). The small sizes are latency-bound: 175 000 small kernel launches and 220 000 stream synchronisations per sweep at $L = 32$.

Table 10: Fitted exponent b of $t \propto L^b$ and local exponents. The fit is a least-squares line through all listed points in log-log space.

Series	Points (L)	Fit b	Local b	Source
CPU, 1 thread	12, 16, 24	6.44	7.0 (12 to 16), 6.1 (16 to 24)	`TUNING_DEFECT.md`
A30, tuned	16, 24, 32, 48	4.45	3.4 (16 to 24), 4.3 (24 to 32), 5.6 (32 to 48)	`TUNING_DEFECT.md`
H100	24, 32	3.58	3.6	`helma_h100.md`
Max 1550 V7	24, 32	4.17	4.2	`ISSUE_INTEL_VS_A30.md`
Max 1550 early	24, 28, 32	4.04	4.4 (24 to 28), 3.6 (28 to 32)	`intel_pvc_supermuc_ng2.md`
MI210	24, 28, 32	4.78	4.5 (24 to 28), 5.1 (28 to 32)	`amd_mi210/README.md`

6.3 Several chains per GPU: MPS, CCS and ranks per GPU

One chain does not fill a GPU. Production therefore places several independent chains (MPI ranks) on one device. Figure 3 shows the aggregate throughput and the time per chain.

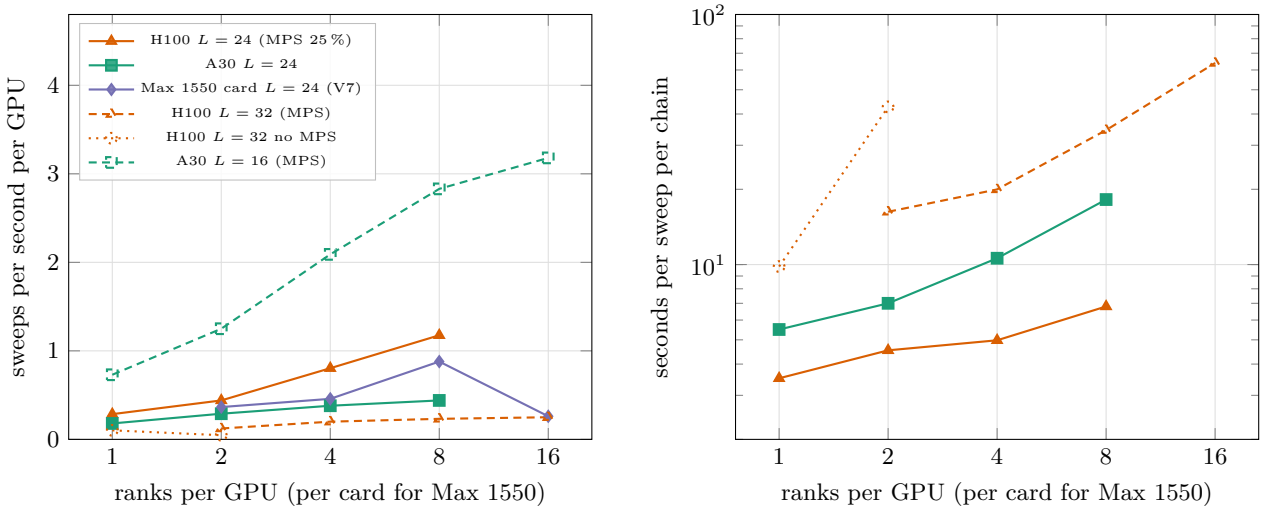


Figure 3: Left: aggregate sweeps per second per GPU against ranks per GPU. Right: time per sweep of each chain. Sources: `helma_h100.md` (H100, A30 $L = 24$), `MPI_GPU.md` (A30 $L = 16$), `ISSUE_INTEL_VS_A30.md` section 7 (Max 1550: 1, 2, 4 and 8 walkers per tile, plotted at 2, 4, 8 and 16 ranks per card). H100 without MPS: 4 and 8 ranks at $L = 32$ did not finish 6 sweeps in 420 s and are not plotted. Whether MPS was active in the A30 $L = 24$ runs is not recorded.

NVIDIA.

- **MPS is mandatory.** Without it, ranks time-slice. On the H100 two ranks are 6.5 times slower per chain at $L = 24$ (29.5 against 4.54 s) and 2.6 times slower at $L = 32$ (42.7 against 16.3 s). Four or eight ranks at $L = 32$ did not finish 6 sweeps in 420 s (`helma_h100.md`). On the A30 older notes give 13 to 15 times (`MPI_GPU.md`). MPS itself costs nothing: one rank under MPS equals one rank without it (`MPI_GPU.md`).
- **H100, $L = 24$:** 8 ranks give 1.176 sweeps per second against 0.285 for one rank, a factor 4.1. Every chain takes 6.80 s instead of 3.51 s (`helma_h100.md`).
- **H100, $L = 32$:** the throughput goes from 0.102 (1 rank, no MPS) to 0.232 (8 ranks) and 0.250 sweeps per second (16 ranks). Going from 8 to 16 ranks gains 6 % and doubles the time per chain (34.5 to 64.0 s). Memory is about 3.2 GB per rank, 51 to 53 GB for 16 ranks. One rank alone reaches an SM utilisation of 55 % (`helma_h100.md`).
- **MPS thread share.** 25 % and 50 % are the same within 2 % at 8 and 16 ranks. 25 % was used in production. With 9 ranks on the A30 (the PP benchmark) the 25 % share was 12 % faster than no cap (`PP_GPU.md`).
- **A30 against H100, 8 ranks, $L = 24$:** 0.44 against 1.18 sweeps per second, a factor 2.7. For one rank the factor is 1.5 to 1.6 (`helma_h100.md`).
- **A30 $L = 16$:** 16 ranks without a cap give 3.18 sweeps per second and 3.7 times longer per chain than one rank. With a 25 % share the aggregate is 4.68 (`MPI_GPU.md`). The A30 is shared and not multiplied: one chain keeps it busy 58 % of the time and two chains 82 %.

Max 1550. There is no MPS on this GPU. Ranks sharing a tile time-slice, and the compute command streamer setting CCS 4 is used in the card layout. In the early build, oversubscribing a tile raised the node throughput by only 0 to 27 % while every chain was 2 to 3 times slower (Table 11). With V7 at $L = 24$ the

throughput per card is 0.365, 0.458, 0.879 and 0.261 sweeps per second for 1, 2, 4 and 8 walkers per tile. So four walkers per tile are the optimum, and eight are 3.4 times worse than four (`ISSUE_INTEL_VS_A30.md`, section 7). **[TODO: cause of the drop at 8 walkers per tile is not documented]**

Table 11: Max 1550, early build (AOT, N_{delay} 128, threads = 112/ranks), one node = 4 cards = 8 tiles, 2 sweeps (`intel_pvc_supermuc_ng2.md`).

L	ranks (per tile)	s/sweep per chain	node sweeps/s	per chain sweeps/s
24	8 (1)	8.25	0.97	0.121
24	16 (2)	17.4	0.92	0.057
24	32 (4)	30.9	1.04	0.032
28	8 (1)	16.2	0.49	0.062
28	16 (2)	32.0	0.50	0.031
28	32 (4)	57.3	0.56	0.018
32	8 (1)	26.3	0.30	0.038
32	16 (2)	48.9	0.33	0.020
32	32 (4)	84.5	0.38	0.012

MI210. The numbers are for separate processes without MPS (`SPEEDUPS.md`): one process 4.46 and 4.52 s per sweep at $L = 24$, four processes 9.16 to 9.27 s each. Four processes deliver about 1.95 times the throughput of one. Pinning four processes to disjoint quarters of the 104 compute units, or two halves shared by two each, does not raise the throughput (Section 6.6).

6.4 Where the time goes

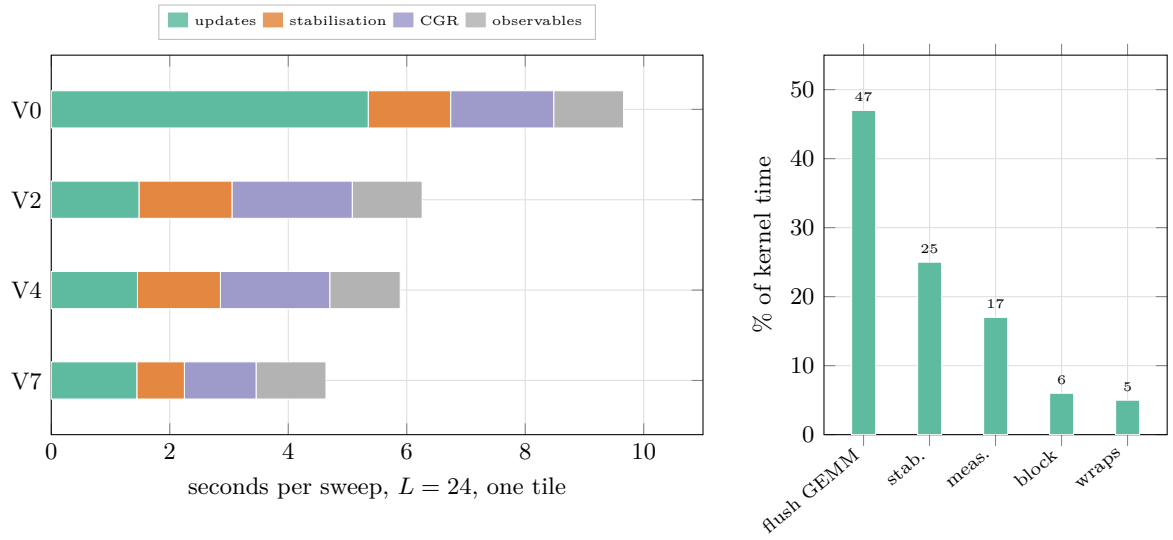


Figure 4: Left: Max 1550, $L = 24$, one chain per tile, wall time per sweep by phase for four variants (`ISSUE_INTEL_VS_A30.md` section 3, `p7/NOTES.md` section 3). The phase times of V0 add up to 9.7 s against a sweep of 10.5 to 10.7 s. Right: A30, 9 ranks with MPS, $L = 24$, share of the kernel time of rank 0 (`PP_GPU.md`).

One chain is latency-bound.

- **A30, $L = 32$, $\beta = 4$, one sweep (nsys):** 44 % of the kernel time are 32 338 small `ZGEMM` launches of 52 μs each (submatrix block flushes), 17 % dense GEMMs and about 9 % QR and LU. The GPU kernel time is 3.8 s of 7.5 s per sweep, and 65 % of the host API time is `cuStreamSynchronize` (`TUNING_DEFECT.md`).
- **Max 1550, $L = 24$, V7:** the device is busy 70 to 78 % of the wall time, but only about one third of the tile's capacity is used. The update phase is bound by host submission: 112 320 asynchronous operations per sweep cost 0.88 s of host time, against 0.56 s of device time in the block path. In the update phase the device is idle 37 % of the time (`p7/NOTES.md`, sections 3, 4 and 6; the busy fraction is an estimate from isolated kernel times).
- **MI210:** the QR panel, which runs in one work-group, is 36 % of the kernel time (`SPEEDUPS.md`).

Production layouts are throughput-bound. On the A30 with 9 ranks under MPS a kernel is running 99.97 % of the time, and on average 5.6 kernels run at once. Flush GEMMs take 47 % of the kernel time of a

rank, stabilisation 25 %, measurements 17 %, block kernels 6 % and wraps 5 % (`PP_GPU.md`). On the Max 1550 with 4 chains per tile the observables take 2.7 s and the updates 2.7 s of 9.3 s per sweep (`SPEEDUPS.md`). Estimated tile utilisation with four ranks is about 69 %, which leaves at most 1.4 to 1.6 times in headroom (`p7/NOTES.md`, estimate).

Table 12: Max 1550, $L = 24$, V7, one chain: the largest device-time items per sweep (`p7/NOTES.md` section 2; times in isolation).

Phase	Item	Calls	Device s	Mean μs
S, C	device QR `geqrf` (sync)	192	0.822	4281
C	`getrf` oneMKL (hybrid host and device)	96	0.379	3943
U	`zgemm` 576^3 (hopping wraps)	2880	0.315	109
U	rank-k `gemm` $576 \times 576 \times k$	34 560	0.312	9.0
O	`corr`, serial kernel (1 thread), first of two	480	0.300	626
S, C	device QR `unmqr`	192	0.262	1367
O	`zgemm` 576^3 (`symm_apply`)	1920	0.206	107
O	`corr`, serial kernel (1 thread), second of two	480	0.166	346

The two serial `corr` kernels (the `acc0` loop over N_{orb}, one thread over 576 sites) cost 0.47 s of 4.9 s per sweep, 9.5 % of a sweep at one chain. The `acc0` reduction was parallelised in the pp-opt line and arrived with the merge, and on the Max 1550 (V7, $L = 24$, one chain) it was measured after the merge: the sweep went from 4.92 to 4.98 s to 4.42 to 4.48 s, about 10 %, with the observables phase from 1.18 to 0.74 s and the other phases unchanged (`CONSOLIDATION.md`). On the A30 the same change gave no speed-up in the 9-rank production layout, because the device throughput is the limit (`PP_GPU.md`).

6.5 Intel Max 1550: the optimisation steps

At the start of the team session one Max 1550 tile ran one chain at 10.5 to 10.7 s per sweep at $L = 24$, about twice the A30 (`ISSUE_INTEL_VS_A30.md`). Each step below is a run-time switch of one binary and was measured by alternating runs in the same job.

Table 13: Steps on Max 1550. Per card: 32 ranks per node, 4 per tile, CCS 4. Ranges are those of the source.

Variant	Switches	$L = 24$	$L = 32$	per card	Numerics
V0	none	10.5 to 10.7	30.3 to 31.7	0.549	reference
V1	staging buffer 1 MB	7.05 to 7.08	23.2	0.568	bitwise identical
V2	V1 + `ALFSH_ASYNCBLK=1`	6.53 to 6.59	22.0 to 22.4	0.655 to 0.688	bitwise identical
V4	V2 + `ALFSH_DEVQR=1`	6.12 to 6.18	19.1 to 19.6	0.737 to 0.785	rounding level
V7	V2 + `DEVQR=2`, `DEVUNMQR=1`	4.95 to 4.97	16.41 to 16.42	0.841 to 0.879	rounding level; $L = 6$, `Mz = .T.` fails 10^{-10}
A30 target		5.25	18.2	0.43	

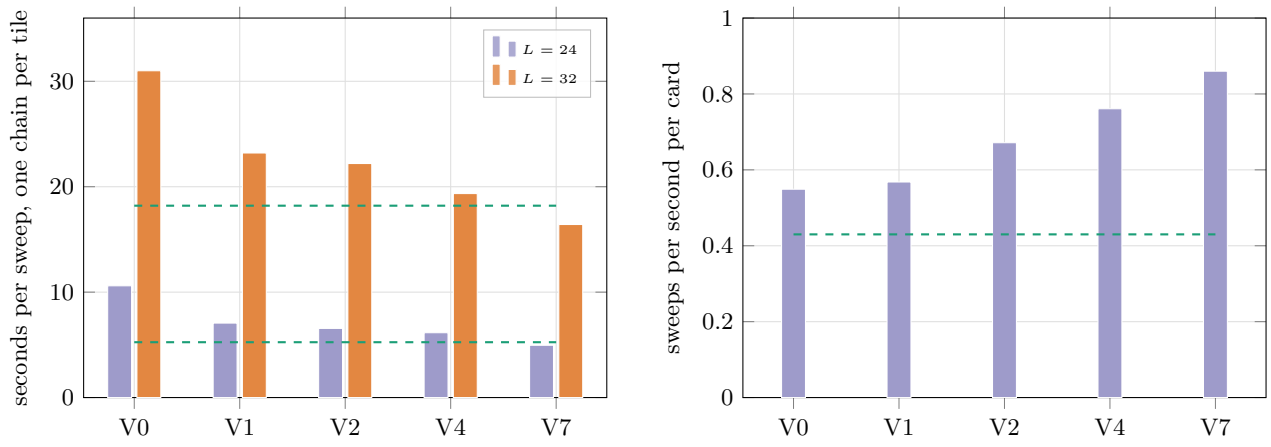


Figure 5: Max 1550 optimisation steps (`ISSUE_INTEL_VS_A30.md`, section 2). Bars are the mid-points of the measured ranges. The dashed lines are the A30 targets: 5.25 and 18.2 s per sweep (left), 0.43 sweeps per second per GPU (right). Per-card values are for the 32-rank layout.

- **V1, staging buffer.** The default Level Zero staging buffer made the 32 KB device-to-host copy of the gathered block cost 380 μs. With a 1 MB buffer it costs 22 μs. There are 8640 such copies per sweep at $L = 24$. This is the largest single lever, $-33\,\%$ at $L = 24$, with bitwise identical results. It is an environment variable, `LIBOMPTARGET_LEVEL_ZERO_STAGING_BUFFER_SIZE=1048576`.
- **Explicit-shape dummies and fused kernel (before V1, not in the table).** First-private arrays fell from 168 800 to 28 544 per sweep and launches from about 75 000 to 49 530, worth 0.4 to 1.2 s at $L = 24$, bitwise identical.
- **V2, async block path.** Block kernels go onto the interop queue without a wait. This gives -0.5 s at $L = 24$, -1 s at $L = 32$ and $+15$ to $20\,\%$ per card.
- **V4, device QR.** The oneMKL `geqrf` is hybrid: the panel runs on the host CPU. Before, this cost 2.4 s per sweep at $L = 24$ and 9.8 s at $L = 32$. The own QR (panel plus blocked update) gives -0.4 s at $L = 24$, -2.6 s at $L = 32$ and $+10$ to $15\,\%$ per card.
- **V7, T-free panel and device apply.** The apply kernel idea came from the AMD work. Per call inside ALF at $L = 24$: `geqrf` 5.48 to 3.91 ms, `unmqr` 2.4 to 1.37 ms. The gate set before the work (`geqrf` at most 0.6 and `unmqr` at most 0.5 times the old time) was missed narrowly (0.61 to 0.71 and 0.52 to 0.71) but the sweep targets are met. V7 helps one chain much more than throughput: $+3.7\,\%$ at 4 walkers per tile, $+16\,\%$ at one.
- **Tried and dropped.** oneMKL batched `geqrf` and `getrf`: no gain. OpenMP `nowait` and `depend`: ifx runs the target tasks undeferred. A batched multi-walker rebuild: the assessment gives a grounded ceiling of 1.5 to 2.3 times, realistic 1.3 to 1.6 times, at about 62 person-days, and a prototype of a batched hopping wrap saves $2\,\%$ only because oneMKL batched `zgemm` is not bitwise equal to the single call. No-go recommended (`ISSUE_INTEL_VS_A30.md`, section 8).

At the start of the session the A30 targets were 5.25 and 18.2 s per sweep for one chain and 0.43 sweeps per second per GPU. V7 meets them: 4.95 to 4.97 s, 16.41 to 16.42 s and 0.84 to 0.88 per card, about twice the A30 per card (`ISSUE_INTEL_VS_A30.md`). Throughput per card is bounded by gemm and rank-k bandwidth.

6.6 AMD MI210

The AMD work removed launch and synchronisation costs in four steps (`amd_mi210/README.md`).

Table 14: MI210, effect of each step. Times as given in the source; the first three rows are CPU time of 2 sweeps, one process.

Step	Effect
Patched libgomp GCN plugin (pooled kernel arguments and signals, pooled small allocations)	empty target region 253 to 37 μs; about 3 times on the real run; `GCN_NUM_THREADS=4` (default 16) was best at $L = 16$
Plugin, threads and first HIP observable kernels	$L = 16$, 2 sweeps: 66 s CPU time with the stock runtime, 21 s after
HIP kernels instead of the hot OpenMP target kernels (a HIP launch and sync is about 15 μs, a libgomp GCN parallel target region about 150 μs)	$L = 16$: 66 to 6 s; $L = 24$: 95 to 19 s; $L = 28$ about 16.6 s per sweep, $L = 32$ about 28.8 s per sweep at that point
Own blocked Householder QR and Q-apply in HIP ($m \leq 2304$)	$n = 576$: `geqrf` 8.8 to 4.4 ms, `unmqr` 8.5 to 1.7 ms. rocSOLVER had made $93\,\%$ of the 2.07 million kernel dispatches per 2 sweeps, $44\,\%$ of the sweep
Deferred synchronisation (presync hook in the plugin)	output bit-identical to the synchronous mode; `ALF_HIP_ASYNC=0` restores it
Result, steady state	$L = 24$: 4.41 s; $L = 28$: 8.84 s; $L = 32$: 17.5 s per sweep

Compute-unit partitioning (`SPEEDUPS.md`, measured 2026-11-13, $L = 24$, steady state). `HSA_CU_MASK` is honoured: 2 of 104 CUs gave 28.4 s per sweep. Results are bitwise identical with and without a mask (25 of 25 output files, also with 4 concurrent processes). It does not raise throughput (Table 15). Recommendation in the source: no masking.

6.7 Cross-platform comparison

Table 16 shows the numbers behind the figure with the ratio to the A30.

The Max 1550 and the MI210 are about as fast as an A30 for one chain. The Max 1550 card reaches twice the A30 throughput only in the layout with four walkers per tile. The MI210 gains nothing from several processes beyond a factor 1.95 over its own single process and ends at the A30 level. The H100 is 2.7 times an A30 for

Table 15: MI210 compute-unit masks, $L = 24$ (`SPEEDUPS.md`).

Configuration	s per sweep per process
1 process, no mask	4.46, 4.52
1 process, quarter of the CUs	about 5.7
4 processes, no mask	9.16, 9.27, 9.21
4 processes, disjoint quarters	9.41, 9.77, 9.42
4 processes, two halves shared by two each	8.74, 8.63 (within noise)

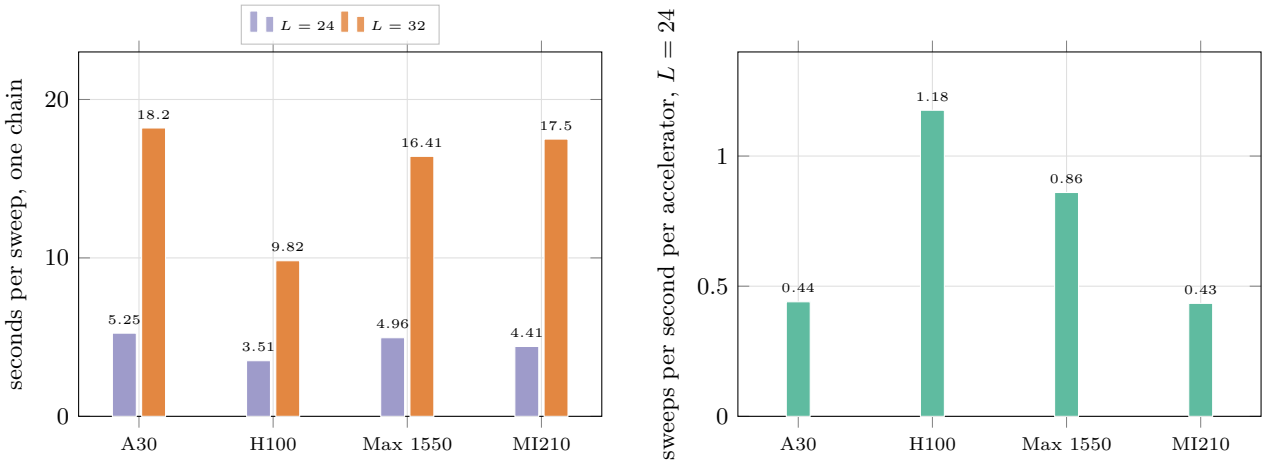


Figure 6: Left: one chain, tuned settings, $L = 24$ and 32 (A30: `TUNING_DEFECT.md`; H100: `helma_h100.md`; Max 1550: V7, `ISSUE_INTEL_VS_A30.md`; MI210: `amd_mi210/README.md`). Right: throughput per accelerator at $L = 24$ for the best measured layout: A30 8 ranks, H100 8 ranks with MPS, Max 1550 card with 8 ranks and CCS 4 (V7, mid-point of 0.84 to 0.88), MI210 4 processes. Throughput at $L = 32$ exists only for the H100 (0.232 sweeps per second at 8 ranks) and is not plotted.

throughput. These comparisons use different layouts and timers, as in Table 9. A comparison of cost or of energy per sweep is not possible from the sources. **[TODO: no cost or energy numbers per sweep exist]**

6.8 Comparison with the CPU

For the defect model the sources give one-thread CPU times on the `nividia` host (same source, `GPU=none`, nvfortran, submatrix `N_delay` 8): 1.68, 12.7 and 148.6 s per sweep at $L = 12$, 16 and 24. The A30 is therefore 1.0, 9.6 and 28 times faster for one chain. At $L = 32$ the source has one 4-thread run at $\beta = 4$ of 107 s per sweep, scaled by 8 to about 860 s for $\beta = 32$, which gives a factor of about 47 against four threads (`TUNING_DEFECT.md`). That value is an extrapolation by the source, not a measurement.

The earlier plain-Hubbard baseline of one rank on the same host (checkerboard, $U = 4$, $\beta = 4$) is 0.60 s per sweep at $N = 144$, 8.0 s at $N = 400$ and 193 s at $N = 1024$, with a 16-core node at 26, 1.8 and about 0.08 chain-sweeps per second (`BENCHMARKS.md`). That model is not the defect model and the dense defect hopping makes the CPU run slower. **[TODO: full-node CPU numbers for the defect model do not exist; a GPU-against-node comparison cannot be made from the sources]**

7 Tuning rules

7.1 Recommended settings

Table 17 is the summary of `TUNING_DEFECT.md`, measured on the A30 on 2026-11-07, with additions from the later platform work.

7.2 `N_delay`

- **A30.** The minimum is at 32 for $L = 16$ and 24 (1.33 and 5.25 s) and at 64 for $L = 32$ (18.2 s, against 18.7 s at 32 and 21.0 s at 128). At $L = 48$ 32 is 1.7 times faster than 8. `N_delay` 0 gives the same time as 64 at $L = 32$ (`TUNING_DEFECT.md`).
- **H100, 8 ranks, MPS, $L = 32$.** 32: 66.5 s per sweep (0.120 sweeps per second); 64: 34.5 s (0.232); 128:

Table 16: Cross-platform comparison. Ratio: A30 time divided by platform time (single chain) and platform throughput divided by A30 throughput.

	s per sweep, one chain		ratio to A30		sweeps/s per device, $L = 24$
Platform	$L = 24$	$L = 32$	$L = 24$	$L = 32$	(ratio)
A30	5.25	18.2	1.00	1.00	0.44 (1.00)
H100	3.51	9.82	1.50	1.85	1.176 (2.7)
Max 1550, V7	4.96	16.41	1.06	1.11	0.86 (2.0)
MI210	4.41	17.5	1.19	1.04	0.434 (1.0)

Table 17: Recommended settings for the defect model on the GPU path.

Parameter	Value	Reason
`Sequential_scheme`	`"submatrix"`	2.4 to 5 times faster than `standard`, 1.4 to 2.1 times faster than `delayed` at the same `N_delay` ($L = 16$ to 32)
`N_delay`	32 for $L \leq 28$, 64 for $L \geq 32$; 0 selects 64	minimum of the scan at $L = 16$, 24, 32 on the A30; Figure 7
`Nwrap`	10; 4 to 5 for `Mz = .T.`	see below
`Checkerboard`	irrelevant	ignored for $h_0 \neq 0$ on CPU and GPU
`Ltau`	0	`SpinZ_localtau` comes from the equal-time pass; `Ltau = 1` works but is slower
`CPU_MAX`	23.8 (hours)	bins are written every 10 sweeps; the run ends at the first bin boundary after `CPU_MAX`, losing at most one bin
Environment	`OMP_TARGET_OFFLOAD=MANDATORY`, `OMP_NUM_THREADS=1` (NVIDIA)	a failed device initialisation otherwise runs silently on the CPU

34.3 s (0.233). So 64 is the choice and 128 gives nothing (`helma_h100.md`).

- **Max 1550, V7,** $L = 24$**.** One chain: 16: 5.72 s, 32: 4.92 (5.01 in an earlier run), 64: 4.73 to 4.74, 128: 5.14. In the layout with 32 ranks: 32: 9.32 to 9.33 s, 64: 9.08 to 9.10 (−2.6 %), 128: 9.50 (`p7/NOTES.md`). In the early build with 14 host threads per rank the time fell from 9.7 to 8.2 s per sweep from 32 to 128 (`intel_pvc_supermuc_ng2.md`). The best value depends on the build.
- **A30, 9 ranks, parameter sets with** $N_{\mathrm{FL}} = 1$**, 6 sweeps.** Wall time 65.8, 57.0, 53.6 and 52.9 s for 16, 32, 64 and 128 (`PP_GPU.md`).

7.3 `Nwrap`

Table 18: `Nwrap` on the A30, $L = 24$, submatrix with `N_delay` 8 (`TUNING_DEFECT.md`).

`Nwrap`	s per sweep	GPU memory (MiB)	Precision Green mean / max
5	11.8	1553	2×10^{-14} / 9×10^{-12}
10	9.9	1069	4×10^{-14} / 1×10^{-11}
20	9.2	825	3×10^{-13} / 5×10^{-11}
40	8.7	703	6×10^{-11} / 1.6×10^{-8}

- `Nwrap` 20 saves 7 % and has a Green precision that is usable. 40 gives 1.6×10^{-8} and is not recommended. `Nwrap` 5 costs 19 % more time and 45 % more memory (`TUNING_DEFECT.md`).
- In the 9-rank A30 production layout `Nwrap` 20 gave 78.5 s against 88.9 s for 10 sweeps. Precision Green mean / max changed from $4 \times 10^{-15}/1.6 \times 10^{-11}$ to $9 \times 10^{-14}/4.4 \times 10^{-10}$ at $U = 3$, and at $U = 8$ from $1.5 \times 10^{-13}/5 \times 10^{-10}$ to $7.6 \times 10^{-13}/4.8 \times 10^{-9}$. Observables agreed to 4×10^{-12} relative after 10 sweeps (`PP_GPU.md`).
- At $L = 32$ `Nwrap` 5 costs about 45 % more than 10. Owner decision 44/45 used `Nwrap` 5. The abort that motivated it was traced to a faulty GPU tile and not to precision (`SPEEDUPS.md`). The ≥ 20 values were measured or rejected earlier and are not re-opened there.
- `Mz = .T.` at $L = 24$ with `Nwrap` 10 has Precision Green mean 8×10^{-8} and max 2×10^{-3}. The stabilisation is not sufficient for the `Mz = .T.` coupling at $U = 5$, so use 4 to 5. Production uses `Mz = .F.` and is not affected (`TUNING_DEFECT.md`).
- The abort threshold of the precision check can be set by `ALF_UNSTABLE_THRESHOLD`. SuperMUC production uses 10^{-4} (`RUNTIME_SWITCHES.md`, `DECISIONS.md` decision 46).

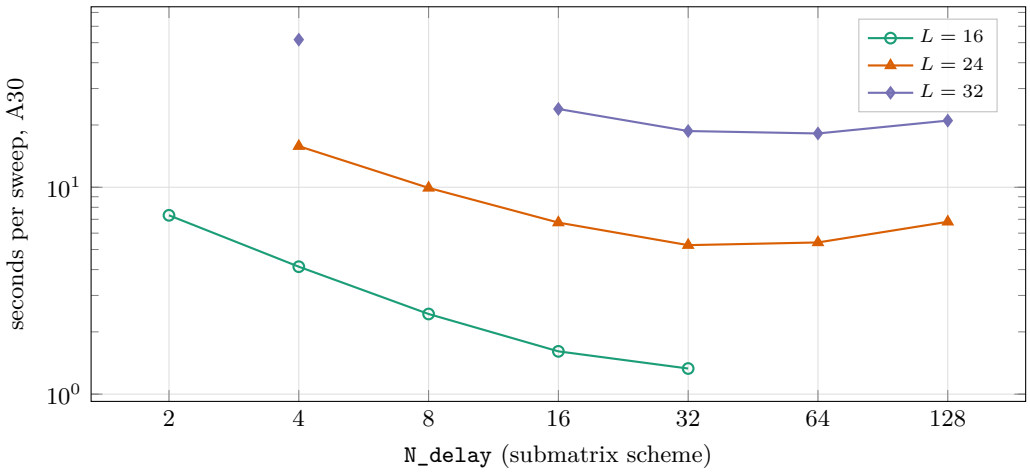


Figure 7: A30, submatrix update, $\beta = L$ (`TUNING_DEFECT.md`). Minimum at 32 to 64. Missing points were not measured.

7.4 Ranks per GPU

Table 19: Layout rules per platform.

Platform	Rule and source
A30	One rank per GPU keeps the time per chain (owner decision 2026-11-08, `MPI_GPU.md`). With MPS 25 % a second rank costs 15 to 17 % per chain at $L = 16$. 4 ranks: 10.5 s per chain at $L = 24$ (0.37 to 0.38 sweeps per second).
H100	8 ranks per GPU with MPS 25 %, `CUDA_MPS_ACTIVE_THREAD_PERCENTAGE=25`, one MPS daemon per GPU. At $L = 32$, 16 ranks give +6 % throughput at twice the time per chain and are not used (`helma_h100.md`).
Max 1550	One rank per tile for the shortest time per chain, four per tile with CCS 4 for throughput (0.84 to 0.88 sweeps per second per card at $L = 24$, V7). Eight per tile is worse (0.26). Bind with `ZE_AFFINITY_MASK=2*local_rank` when there are fewer than 8 ranks per node (`ISSUE_INTEL_VS_A30.md`, `intel_pvc_supermuc_ng2.md`).
MI210	No masking. Several separate processes: 4 give about 1.95 times one (`SPEEDUPS.md`).

Cost of the layout choice on Max 1550. For a scan of 10 U values with 16 ranks per set, more ranks per tile save 8 to 20 % of the GPU hours but stretch the wall time by a factor 2 to 3.5 (`intel_pvc_supermuc_ng2.md`). At $L = 32$ with $S = 5000$ usable sweeps per U point, 1, 2 and 4 ranks per tile need 769, 713 and 616 GPU-h on 20, 10 and 5 nodes. The warm-up is 62 to 76 % of every chain in these estimates (assumption of 1000 warm-up sweeps).

Production layout on the H100 (Helma, as submitted 2026-11-11). Ten U values, 2 parts per value, 8 walkers per part, one GPU per part, 20 independent job chains of 24-hour links in the `h100` partition. Worst case 1920 GPU-h. The projection of the source is 0.232 sweeps per second per GPU, 20 000 sweeps per GPU per day. This is arithmetic on Section 6.3, not a production measurement. At the time the note was written (2026-11-11 15:20) no production link had started. **[TODO: production status of the Helma chains after 2026-11-11 is not in the sources]**

8 Open speedup candidates

State of `SPEEDUPS.md`, 2026-11-13. The rule that organises the list: a change that removes launches helps single chains. Only a change that removes device work helps production layouts, which are throughput-bound (Section 6.4).

8.1 Available now, no code change

8.2 Implemented on `speedup/nvidia`, not yet measured

The A30 was occupied by the parameter-set production during this work. The two changes compile with nvfortran 26.9 but have neither timing nor a numerics run. The script `~/alf-gpu/work/spd1/ab.sh` on `nividia` runs the checks once the GPU is free (`SPEEDUPS.md`). **[TODO: timing and numerics of commits a3f257c6 and 50f837f7]**

Table 20: Switches that need no code change (`SPEEDUPS.md`).

Item	Gain	Numerics	State
Intel: `LIBOMPTARGET_LEVEL_ZERO_STAGING_BUFFER_SIZE=1048576`	$L = 24$, one chain: 10.6 to 7.1 s per sweep	bitwise identical	Set in the team jobs. Not found in `prod/job_preempt.sh`. Owner decision for new production sets; a running set keeps its settings.
Intel: `ALFSH_ASYNCBLK=1`	$L = 24$ about 0.5 s per sweep, 15 to 20 % per card	bitwise identical	Waits for the M12 timing test.
All: `ALF_GPU_OBS_SKIP` (no `Green_eq`, `Den_eq`)	7 % at $L = 24$ under MPS	skipped outputs stay zero	Needs `ALF_GPU_DEBUG=1` today. Owner decision whether the analysis needs these outputs.
`Nwrap` 10 instead of 5 at $L = 32$	`Nwrap` 5 costs about 45 % more	precision rule of the reference parameter set	Owner decision 44/45 used 5; the motivating abort was a faulty GPU tile.
Intel: parallel `acc0` reduction in `corr` (came with the merge from pp-opt)	the two serial `corr` kernels were 0.47 s of 4.95 s at $L = 24$	summation order of `acc0` changes (10^{-13})	In `gpu-port-unified`. Measured on SuperMUC-NG: $L = 24$ one chain V7 4.92 to 4.98 → 4.42 to 4.48 s per sweep, about 10 % (`CONSOLIDATION.md`).

Table 21: Not yet measured changes (`SPEEDUPS.md`).

Commit	Change	Expected gain and claim to verify
`a3f257c6`	Phase of the determinant without an LU. In `dev_cgr` the `la_getrf` of $U_R^H U_L$ is used only by `k_det_phase`. The phase follows from the Householder scalars, $\det H_i = -\tau_i/\overline{\tau_i}$, tracked per stack slot in `dev_decompose`. `ALF_GPU_CGR_LU=1` restores the LU, `=2` runs both.	1.2 % per sweep at $L = 32$ single chain, about 3 % at $L = 24$ in the A30 MPS production (from profiles). Larger on Intel, where `getrf` is host-assisted (0.38 s of 4.9 s, about 8 %). G bitwise unchanged; phase differs at rounding level.
`50f837f7`	`la_sync` in `blk_end` removed on CUDA.	10 to 20 ms per sweep, bitwise identical.

8.3 Measured and decided

- **Port the AMD device QR to CUDA: no, for now.** On the A30, single chain, cuSOLVER `geqrf` takes 7.8 ms at $n = 1024$, `ormqr` 2.2 ms and `getrf` 4.1 ms, together 8 % of a sweep. The AMD gain was against hipSOLVER, which is much slower. Best case about 3 % per sweep for a single chain, at most 8 % per rank in production. Worth another look only if single-chain `geqrf` at $n = 576$ is well above 4 ms.
- **Fused block-end kernel (AMD b3923afe): no gain** ($L = 24$ stays at 4.41 s). Not ported.
- **`N_delay` 128, `Nwrap` $\geq$ 20, oneMKL batched routines, `nowait` on ifx, CholeskyQR2, `STABLOG`:** measured or rejected earlier, not re-opened.
- **MI210 compute-unit masks:** no throughput gain (Section 6.6).

8.4 Larger candidates, ranked by expected effect on production throughput

These are design notes without code (`SPEEDUPS.md`).

The Intel kernel study adds two throughput levers that are not in the list above (estimates, not tried; `p7/NOTES.md`): a better mapping of the large `corr` kernels (about +5 to 8 % throughput at 4 ranks per tile) and fewer, cheaper host submissions in the update block path (pack the 4 uploads per block, fuse the 4 rank-k `gemm`, or reuse a command list; −0.4 to −0.5 s per sweep for one chain, +5 to 8 % throughput). The two serial `corr` kernels are already addressed by the merged parallel reduction.

9 Limitations and open decisions

9.1 Limitations

Scope. Only the predefined Hubbard model with diagonal rank-1 vertices runs on the device. Everything in the refused rows of Table 6 runs on the CPU. Real parallel tempering, projector, Langevin, HMC, MALA, cross-slice delay and time-dependent couplings of `Op_V` are refused. Time-displaced observables with the defect use the host `ObserT`. Custom observables and entanglement are not implemented.

Size.

Table 22: Larger candidates in the order of the source.

#	Idea	Mechanism	Numerics	Effort
1	Two-level submatrix update (`blk_end`, `blk_flush`)	Keep the pending low-rank update XY^H over all blocks of a time slice and correct each block's gather by $X(B,:)Y(B,:)^H$. G is flushed once per slice with rank about 190 instead of once per block with rank 10 to 21. The $k \times k$ algebra has to move to the device. Targets flush GEMMs, 47 % of kernel time in the A30 production.	rounding changes	large (weeks)
2	Static hopping by FFT or band (`Hd`, `symm_apply`)	The static `Op_T` is a dense $N \times N$ exponential of a translation-invariant matrix. An FFT or a truncated band removes about 5800 dense GEMMs per sweep at $L = 24$.	rounding changes; truncation must be shown to be below one ulp	medium to large; FFT per backend
3	`corr` with coalesced memory access	Threads read G along shifted diagonals. A pre-pass with a tiled transpose, then binning by distance, removes uncoalesced loads. On Intel the large `corr` kernels are 3.5 times slower than their data volume.	summation order	medium
4	Equal-time measurements on fewer slices	Measuring every second slice halves 12 to 25 % of a sweep.	no bias, larger variance per sweep; owner decision on comparability	small
5	QR panel look-ahead (`alf_hip_qr.hip.cpp`, `alf_la_sycl_qr2.hpp`)	Factor panel $j+1$ while the trailing update of panel j runs. Helps single chains on AMD and Intel.	same arithmetic	medium; node race test again
6	Device `getrf` on Intel	oneMKL `getrf` is host-assisted (13 ms at $N = 1024$). Commit `a3f257c6` removes half of the calls.	rounding changes	medium

- $L = 64$ ($N = 4096$) cannot be started on any backend. `Op_set` stops with "Eigenvector matrix has near-zero determinant" because `Det_C` of the eigenvector matrix underflows. A fix has to compute the determinant phase without forming the product of 4096 LU diagonal elements. This is outside the port and not changed (`TUNING_DEFECT.md`).
- Memory grows as $\approx (4N_{\mathrm{STM}} + 40) \cdot 16N^2$ bytes. $L = 48$ with $\beta = 48$ needs 19.6 GB.

Verification.

- The consolidated branch is verified on the CPU path, on the MI210 and on the Max 1550. The A30 and H100 device runs are **[TODO: pending]** (Section 5.3).
- The CI builds and tests the CPU path only (`README.md`).
- The default path on the A30 was not bitwise reproducible on 2026-10-29 (Section 5).
- The $L = 6$, `Mz = .T.` tolerance is noise-limited. oneMKL itself fails it for 2 of 5 seed sets.
- Not tested, from `TIMEDEP_HOPPING.md` (state 2026-11-07): `SELFTEST` 3 and 4, $L = 32$ with `Mz = .T.` and `Ltau = 1`, runs longer than 2 sweeps at $L \geq 32$ against the CPU, and `Nwrap` $\neq 10$ against the CPU. **[TODO: check which of these were closed later]**
- Whether the result is bitwise independent of some tuning variables is not determined in the code review: `ALF_GPU_TL`, `ALF_GPU_TR`, `ALF_GPU_VB` and `ALF_HIP_QR` (`RUNTIME_SWITCHES.md`).

Intel device QR.

- A race in the first version of the device QR gave wrong results with 4 ranks per tile under CCS 4 (Precision Green maximum 0.55 to 1.96). The cause was a tau buffer reused two panel steps early. It was fixed with three slots and a device-limit check (`ISSUE_INTEL_VS_A30.md`).
- The node race test found 0 bad results in 0.85 million iterations at $N = 1024$ and 3.39 million at $N = 576$ on two nodes. In the planted-defect controls a one-buffer defect was caught at once, an exchange-slot defect rarely (4 in 337 000), a v-buffer defect only with an amplifier setting, and a tau-slot defect never in 2.2 million iterations. Races of 10^{-5} per iteration or more are excluded (95 %: below 3.5×10^{-6} at $N = 1024$). A tau-type race, the class of the earlier bug, would be missed. A code review of the tau-slot logic is not done. A fresh numerics review of the device QR before production is open.

Point-defect observables. With `ham_h0` $\neq 0$ the code now refuses `Projector`, Langevin and HMC updates, and a restricted measurement window (`LOBS_ST`, `LOBS_EN`). `SpinZ_local` and `SpinZ_localtau` are normalised only for finite temperature, sequential updates and all slices. Known defects outside that setting

(CONSOLIDATION.md): SpinZ_localtau is NaN if slice $L_{\mathrm{trot}}/2$ is never measured; with the projector the endpoint weights are wrong and the pin lies inside the projection region; with Langevin or HMC the endpoint weights are wrong. A proper fix needs the same change in the host code, gpu_obs_mod.F90 and the HIP kernel and is open. With Ltau = 1 the pinning field is switched off (ham_h0 = 0), and info says so. The site formula of the defect assumes a square lattice.

Hardware and tools.

- On SuperMUC-NG Phase 2 a faulty GPU tile caused sporadic production aborts ("G difference > 10"). It was diagnosed with a stress test and reported to LRZ (ISSUE_INTEL_VS_A30.md).
- On that system VTune 2025.1 and 2025.8 gpu-offload collect no GPU data, Level Zero tracing tools crash ALF at start-up, and Intel Advisor failed (p7/NOTES.md). Occupancy figures are estimates from launch geometry.
- The AMD speed needs a patched libgomp GCN plugin. Without it every HIP call synchronises (amd_mi210/README.md). The patches apply to GCC 14.2.0.
- nvfortran 26.9 mis-compiles blk_end at -O2 and above, pinned to -O1. The cause is not understood (DELAYED_SUBMATRIX.md).

Performance data.

- The numbers have different timers and run lengths (Table 9). The H100 benchmarks are 6 sweeps long, so a warm-up or one outlier shows. The full 4-GPU node and finer GPU utilisation were not measured (helma_h100.md).
- There is no multi-rank A30 number for $L = 32$ and no CPU full-node number for the defect model.
- Single-chain fits of the scaling exponent use two to four points.

Production budget on Intel. With the budget of the project alone (840 of 900 GPU-h) the best layout for both lattice sizes reaches only 1000 usable sweeps per U point, which is nothing beyond the warm-up (intel_pvc_supermuc_ng2.md). Options named there: more budget, one lattice size at a time (517 GPU-h for $L = 28$ at $S = 10\,000$), fewer U values, or warm-starting from confout of a shorter run.

9.2 Open decisions

Table 23: Decisions that belong to the owner.

#	Decision	Source
1	Add the 1 MB staging buffer to the production job script of Intel runs. It is environment-only and bitwise identical in tests. The source recommends a short same-seed check of the production binary with and without it.	SPEEDUPS.md, ISSUE_INTEL_VS_A30.md
2	Release ALFSH_ASYNCBLK=1 (V2) after the M12 timing test and a same-seed bitwise check.	SPEEDUPS.md
3	Release V4 and V7: needs the numerics review of the device QR (V4), a new pre-registered rule for the $L = 6$, Mz = .T. test and a review of the tau-slot logic (V7).	ISSUE_INTEL_VS_A30.md
4	Allow ALF_GPU_OBS_SKIP in production, if the analysis does not need Green_eq and Den_eq. Gain 7 % at $L = 24$.	SPEEDUPS.md
5	Nwrap 10 or 5 at $L = 32$.	SPEEDUPS.md
6	Measure on every second slice (halves 12 to 25 % of a sweep) given the loss of comparability with existing data.	SPEEDUPS.md
7	Start the two-level submatrix update (weeks of work; targets the largest item in production).	SPEEDUPS.md
8	Fix of the determinant in Op_set for $L = 64$.	TUNING_DEFECT.md
9	Budget and layout of the Intel scan (Section on production budget above).	intel_pvc_supermuc_ng2.md
10	Not merged on purpose: the variant branch with the older tau labelling of SpinZ_localtau (...-old-Ntau). It must not flow into production.	CONSOLIDATION.md

A Build lines per platform

Sources: README.md, BUILD.md, helma_h100.md, intel_pvc_supermuc_ng2.md, amd_mi210/README.md. After configure.sh the build is make lib program.

```
# CPU reference (unchanged upstream code)
```

```
source configure.sh GNU MPI HDF5                # or the compiler of the GPU build

# NVIDIA A30 (nvfortran 26.9, MACHINE PGI)
source configure.sh PGI MPI GPU=nvidia HDF5 NO-INTERACTIVE
source configure.sh PGI PP  GPU=nvidia HDF5 NO-INTERACTIVE   # parameter sets
# default ALF_GPU_CC=cc80

# NVIDIA H100 on Helma (nvhpc 25.7; login shell, module command)
export ALF_HDF5_DIR=/hnvme/workspace/b133ae29-ALF_GPU/hdf5_257
. ./configure.sh HELMAGPU PP GPU=nvidia HDF5 NO-INTERACTIVE
# loads gpu-env/2025, nvhpc/25.7, openmpi/5.0.8-nvhpc25.7-cuda; flags cc90,cuda12.9

# Intel Max 1550 on SuperMUC-NG Phase 2 (intel-toolkit/2025.1.0)
source configure.sh IntelLLVM MPI GPU=intel HDF5
# AOT for PVC through scripts/supermuc_ng2/build.sh:
AOT=1 bash -l build.sh prodaot IntelLLVM MPI GPU=intel HDF5 NO-INTERACTIVE
# ifx wrapper rewrites -fopenmp-targets=spir64 to spir64_gen -device pvc
# (start-up of one process: JIT about 65 s, AOT 14 s)
# SYCL libraries: ALF_SYCL_BLK_FLAGS='-fsycl-targets=spir64_gen -Xs "-device pvc"'

# AMD MI210 (gfortran 14 with amdgcn offload, ROCm)
source configure.sh GNU MPI GPU=amd HDF5
# ALF_AMD_ARCH=gfx90a (default), ROCM_PATH=/opt/rocm, hipcc for the kernels
# patched libgomp GCN plugin first in LD_LIBRARY_PATH (see amd_mi210/README.md)
```

The `GPU=none` build adds no flag and no object code. The identity of the exported variables with upstream was checked for GNU, IntelLLVM and PGI with and without MPI, `LOG`, HDF5 and `Devel` (`BUILD.md`).

B Environment switches that matter in production

The full list with categories is in `RUNTIME_SWITCHES.md`. The production guard stops a run if any `ALF_GPU_*` or `ALF_LA_TEST_*` variable except `ALF_GPU_DISABLE` and `ALF_GPU_DEBUG` is set. This includes a leaked build variable such as `ALF_GPU_CC` (from the code, not tested). The variables `ALFSH_*`, `ALF_HIP_*` and `ALFDEVQR_*` are not guarded and have to be checked by hand.

Source documents

Everything in this report comes from the following files in `docs/gpu/`: `README.md`, `CONSOLIDATION.md`, `SPEEDUPS.md`, `RUNTIME_SWITCHES.md`, `FEATURES.md`, `NOT_IMPLEMENTED.md`, `ADR-001-programming-model.md`, `ADR-002-alf-la-and-data-model.md`, `DELAYED_SUBMATRIX.md`, `TIMEDEP_HOPPING.md`, `TUNING_DEFECT.md`, `MPI_GPU.md`, `PP_GPU.md`, `VERIFICATION.md`, `BENCHMARKS.md`, `BASELINE.md`, `PROFILE.md`, `BUILD.md`, `HARD.md`, `platforms/helma_h100.md`, `platforms/intel_pvc_supermuc_ng2.md`, `platforms/amd_mi210/README.md`, `sessions/supermuc_large_prompt/ISSUE_INTEL_VS_A30.md` and `sessions/supermuc_large_prompt/phase2/team/p7/NOTES.md`, plus the verification runs of 2026-11-13 reported by the owner.

Table 24: Variables used in production job scripts.

Variable and value	Platform	Effect
`OMP_TARGET_OFFLOAD=MANDATORY`	all	Fail if no device is found instead of running on the host.
`OMP_NUM_THREADS=1`	NVIDIA	One host thread per rank (`helma_h100.md`, `MPI_GPU.md`).
`ALF_GPU_DISABLE=1`	all GPU	GPU build runs the host path. Allowed in production.
`ALF_UNSTABLE_THRESHOLD`	all	Abort threshold of the precision check, default 10. SuperMUC production uses 10^{-4}.
`CUDA_MPS_ACTIVE_THREAD_PERCENTAGE=25`, `CUDA_MPS_PIPE_DIRECTORY`, `CUDA_MPS_LOG_DIRECTORY`	NVIDIA	One MPS daemon per GPU; SM share per client. Mandatory for several ranks per GPU.
`CUDA_VISIBLE_DEVICES` (through `rank_wrap.sh`)	NVIDIA	Binds rank r to GPU r/RPG (`helma_h100.md`).
`LIBOMPTARGET_LEVEL_ZERO_STAGING_BUFFER_SIZE=1048576`	Intel	V1, $-33\,\%$ at $L = 24$ one chain, bitwise identical.
`ALFSH_ASYNCBLK=1`	Intel	V2, async block path, bitwise identical, +15 to 20 % per card. Not yet released.
`ALFSH_DEVQR=1`, `=2`, `ALFSH_DEVUNMQR=1`	Intel	V4 and V7 device QR and apply. Rounding level; not released.
`ZE_FLAT_DEVICE_HIERARCHY=FLAT`	Intel	One tile is one device, 8 per node; rank r uses tile r mod 8.
`ZE_AFFINITY_MASK`, `ONEAPI_DEVICE_SELECTOR=level_zero:N`	Intel	Select tile or device; bind with `2*local_rank` for fewer than 8 ranks per node.
`I_MPI_PIN_DOMAIN=auto`	Intel	`omp:compact` was 35 % slower than `auto` at one rank per tile.
`ALF_HIP_QR=0`, `ALF_HIP_ASYNC=0`	AMD	Switch off the own QR or the deferred synchronisation (default on, async only with the patched plugin). Output bit-identical for async; checked bitwise on 2026-11-13 (Table 8).
`GCN_NUM_THREADS=4`	AMD	Threads of the GCN OpenMP runtime; 4 was best at $L = 16$.
`HSA_CU_MASK`	AMD	Honoured, bitwise neutral, no throughput gain. Not recommended.
`mpirun -bind-to core`	all	One core per rank. On Helma also `-oversubscribe` and `-mca smsc ^knem`.